

Smart Real Estate Investment Advisor

Yoetz Nadlan Chacham

ML-Powered Real Estate Analytics for the Israeli Market

Course:	Mathematical Geodesy - Ariel University
Dataset:	10,424 Transactions Israel Tax Authority
Model:	XGBoost $R^2 = 0.741$ RMSE ~ 607,000 ILS
Stack:	Python Streamlit scikit-learn XGBoost
Date:	May 2026

1. Project Overview

The Smart Real Estate Investment Advisor is a data-driven platform designed to help Israeli real estate investors identify promising settlements and undervalued deals. It combines transaction data from Israel's Tax Authority with socioeconomic indices from the Central Bureau of Statistics (CBS/LAMAS), trains a machine learning model to predict fair apartment prices, and serves a Streamlit web application that scores and ranks settlements based on the investor's profile.

2. The Core Idea

The platform is built around one central insight: if the ML model predicts an apartment should sell for X, but it actually sold for less - that is a potential investment opportunity. By aggregating this 'price gap' across all deals in a settlement, combined with price trends and market liquidity, the app produces a Viability Score (0-100) for each settlement.

Viability Score Formula

Current Yield goal: 60% price gap + 20% price trend + 20% liquidity

Appreciation goal: 30% price gap + 50% price trend + 20% liquidity

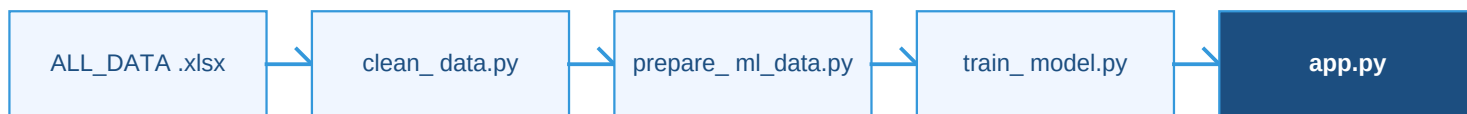
Positive gap = property sold below model prediction = potential opportunity

3. Tech Stack

Layer	Technology	Purpose
Data Collection	Python + requests + pyproj	GovMap API scraping + coordinate transforms
Data Processing	pandas + numpy	Cleaning, filtering, feature engineering
Machine Learning	XGBoost + scikit-learn	Price prediction model
Model Storage	joblib	Serialize & load model.pkl
Web App	Streamlit	Interactive investment dashboard
Socioeconomic Data	CBS / LAMAS	Settlement-level index enrichment

4. Data Pipeline

The data flows through four sequential stages before reaching the model and app:



Stage 1 - nadlan_loader.py: Data Collection

Scrapes GovMap API (govmap.gov.il) across 20 ITM coordinate bounding boxes covering Israel.

Collects up to 10,000 transactions: max 80 per city, max 3 per building.

Guarantees at least 25% of each city's deals are from 2025+ (recency requirement).

Converts coordinates: ITM (EPSG:2039) <-> Web Mercator (EPSG:3857) using pyproj.

Output: raw deal data with polygon IDs, deal dates, prices, and geocoordinates.

Stage 2 - clean_data.py: Outlier Removal

Loads ALL_DATA (1).xlsx from the Israel Tax Authority.

Removes deals with price < 100,000 ILS or > 10,000,000 ILS.

Removes properties with area < 20 m2 or > 500 m2.

Output: all_data_clean.csv

Stage 3 - prepare_ml_data.py: Feature Engineering

Filters out non-residential types: offices, stores, warehouses, industrial, agricultural, etc.

Parses dealDate into deal_year and deal_month numeric features.

Maps Hebrew floor names to integers (basement=-1, ground=0, first=1, ...).

Fills missing rooms / floor / socio values with column medians.

Label-encodes settlement, deal nature, and neighborhood as category codes.

Target-encodes street names: each street replaced by its mean sale price.

Saves apartments_ml_ready.csv (for model) and apartments_display.csv (for UI).

5. Socioeconomic Data - LAMAS_EDA_2M

Israel's Central Bureau of Statistics (CBS) publishes a socioeconomic index for every locality, ranking it across 10 clusters based on income, education, employment, and demographics. This index is merged into every transaction as the feature 'socio_index_avg', and is used in the app to separate established markets (high index) from emerging ones.

Part	Script	Purpose	Output
Part 1	gen_eda.py	Interactive EDA dashboard	eda.html, data_preview.html
Part 1	gen_eda_app.py	App-style EDA with cluster filter	eda_app.html
Part 2	clean_missing.py	Impute missing values (mean/mode)	-
Part 2	check_duplicates.py	Identify and report duplicates	missing_report.html
Part 3	gen_eda_final.py	Final cleaned analysis	DATA_final.csv, eda_final.html

Features in the socioeconomic dataset include: socioeconomic cluster (1-10), average schooling years, % with academic degree, employment rate, income distribution percentiles, average days abroad, vehicles per 100 residents, and age/dependency ratios.

6. Bat Yam Case Study - DATA_FILES/BATYAM_2M

Before scaling the pipeline to all of Israel, Bat Yam was used as a single-city prototype to validate the entire workflow end-to-end: data collection, cleaning, EDA, and analysis.

File	Purpose
clean_bat_yam.py	Filters nadlan_final.csv to Bat Yam only, cleans and reports
gen_bat_yam_eda.py	Generates EDA HTML report for Bat Yam transactions
gen_nadlan_preview.py	HTML preview of the raw nadlan data
analyze_area.py	Area-level statistics and analysis
bat_yam_clean.csv	Cleaned Bat Yam transaction dataset
bat_yam_eda.html	Interactive EDA dashboard for Bat Yam

7. Machine Learning Model - train_model.py

An XGBoost Regressor is trained to predict apartment sale prices (dealAmount) from engineered features. The model is serialized to model.pkl and loaded at app startup to run predictions on all apartments in the dataset.

5,287 Training rows <i>80% split</i>	1,322 Test rows <i>20% split</i>	0.741 R2 Score <i>explains 74% of variance</i>	607K ILS RMSE <i>~35% of mean price</i>	500 Trees <i>depth 6, lr 0.05</i>
---	---	---	--	--

XGBoost Hyperparameters

n_estimators = 500 | Number of boosting trees
learning_rate = 0.05 | Step size - slow but stable convergence
max_depth = 6 | Tree depth - balances fit and overfitting
subsample = 0.8 | 80% of rows sampled per tree
colsample_bytree = 0.8 | 80% of features sampled per tree
n_jobs = -1 | Uses all available CPU cores

8. Key Features Used by the Model

Feature	Type	Description
assetArea	Numeric	Apartment area in m2
assetRoomNum	Numeric	Number of rooms
floor_num	Numeric	Floor number (Hebrew names mapped to int)
deal_year	Numeric	Year of transaction
deal_month	Numeric	Month of transaction
settlement_encoded	Category code	Settlement / city (label encoded)
neighborhood_encoded	Category code	Neighborhood within city (label encoded)
deal_nature_encoded	Category code	Deal type (apartment type)
street_price_mean	Target encoded	Mean sale price on this street
socio_index_avg	Numeric	CBS socioeconomic index of the settlement
socio_rank_avg	Numeric	CBS socioeconomic rank of the settlement
X, Y	Numeric	ITM spatial coordinates

9. Web Application - app.py

The Streamlit app provides an interactive investment dashboard with two tabs. It loads the trained model at startup and computes settlement-level statistics by running predictions across all apartments in the dataset.

How the App Works (Mode A - Area Recommendations)

1. Load model.pkl and run predictions on apartments_ml_ready.csv
2. Calculate gap_pct = (predicted - actual) / actual x 100 per apartment
3. Compute annual price trend per settlement using linear regression on deal_year
4. Aggregate: avg price, avg gap, deal count, avg socio index per settlement
5. Filter by: max budget, risk level (socio above/below median), min deal count
6. Score each settlement 0-100 using weighted combination of gap, trend, liquidity
7. Display top 15 settlements ranked by viability score

10. User Profile Inputs

Input	Options	Effect on Results
Max Budget (ILS)	300K - 10M (step 100K)	Filters out settlements above avg price
Investment Goal	Current Yield / Appreciation	Changes score weights (gap vs trend)
Risk Level	Established / Emerging	Filters by socio index above/below median
Min Deals (liquidity)	5 - 50	Minimum transaction count threshold

11. Score Weights by Investment Goal

Component	Current Yield Weight	Appreciation Weight
Price Gap (predicted vs actual)	60%	30%
Price Trend (% change per year)	20%	50%
Liquidity (deal count)	20%	20%

12. Complete File Structure

File / Folder	Role
app.py	Streamlit web app - main entry point
train_model.py	Trains XGBoost model, saves model.pkl
model.pkl	Serialized trained model (~1.7 MB)
requirements.txt	Python dependencies
run_app.bat	Windows shortcut to launch the app
DATA_FILES/ALL_DATA (1).xlsx	Raw source from Israel Tax Authority
DATA_FILES/nadlan_loader.py	GovMap API scraper - data collection
DATA_FILES/clean_data.py	Removes outliers from raw Excel
DATA_FILES/prepare_ml_data.py	Full feature engineering pipeline
DATA_FILES/nadlan_final.csv	10,424 cleaned transactions
DATA_FILES/all_data_clean.csv	Price + area filtered dataset
DATA_FILES/apartments_ml_ready.csv	Apartment features ready for XGBoost
DATA_FILES/apartments_display.csv	UI-facing columns for the app
DATA_FILES/ISRAEL_POINTS_FILTERED_GEO.csv	Geographic reference / POI data
DATA_FILES/BATYAM_2M/	Bat Yam single-city prototype scripts
LAMAS_EDA_2M/PART1/	CBS socioeconomic EDA + dashboards
LAMAS_EDA_2M/PART2/	Missing values + duplicate handling
LAMAS_EDA_2M/PART3/	Final cleaned socioeconomic dataset

13. Project Status

Feature	Status	Notes
Data collection (GovMap scraper)	Done	10K deals across 20 regions
Data cleaning pipeline	Done	Price, area, type filters
Feature engineering	Done	12+ ML-ready features
Socioeconomic enrichment (LAMAS)	Done	Merged into every transaction
XGBoost model training	Done	R2=0.741, RMSE=607K ILS
Mode A - Area Recommendations	Live	Fully working in Streamlit
Mode B - Property ranking in region	Planned	Not yet implemented
Mode C - Single deal evaluation	Planned	Not yet implemented
Mode D - Side-by-side comparison	Planned	Not yet implemented
GIS map visualization	Planned	Folium/Leaflet mentioned

Generated automatically with Claude Code | May 2026